



به نام خدا



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر
مدل‌های مولد عمیق
دکتر توسلی‌پور
تمرین چهارم

نام و نام خانوادگی	علیرضا نجفی مطیعی
شماره دانشجویی	۸۱۰۱۰۰۲۲۴
تاریخ ارسال گزارش	۱۴۰۴.۱۱.۲۷

فهرست

سؤال اول: مدل‌های دیفیوژن.....	۳
بخش اول: بخش تئوری.....	۳
بخش دوم: بخش عملی.....	۷
سوال دوم: فلومچینگ و سری‌های زمانی.....	۱۶
بخش اول: سوالات تئوری.....	۱۶
بخش دوم: سوالات عملی.....	۱۸
مراجع.....	۲۵

سؤال اول: مدل‌های دیفیوژن

بخش اول: بخش تئوری

سؤال اول: اثبات فرم بسته

برای اثبات از تکنیک Reparametrization Trick و ویژگی جمع‌پذیری توزیع‌های گوسی استفاده می‌کنیم:

$$q(x_t|x_{t-1}) = N(\sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t)I)$$

$$\rightarrow x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{(1 - \alpha_t)}\epsilon_{t-1}, \epsilon_{t-1} \sim N(0, I)$$

$$\rightarrow x_{t-1} = \sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{(1 - \alpha_{t-1})}\epsilon_{t-2}$$

$$\rightarrow x_t = \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{(1 - \alpha_{t-1})}\epsilon_{t-2}) + \sqrt{(1 - \alpha_t)}\epsilon_{t-1}$$

$$\rightarrow x_t = \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{\alpha_t(1 - \alpha_{t-1})}\epsilon_{t-2} + \sqrt{1 - \alpha_t}\epsilon_{t-1}$$

عبارت‌های دوم و سوم درواقع ترکیب دو نویز گوسی هستند پس برای واریانس نویز جدید داریم:

$$var = (\sqrt{\alpha_t(1 - \alpha_{t-1})})^2 + (\sqrt{1 - \alpha_t})^2 = \alpha_t(1 - \alpha_{t-1}) + 1 - \alpha_t = 1 - \alpha_t\alpha_{t-1}$$

$$\rightarrow x_t = \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_t\alpha_{t-1}}\bar{\epsilon}$$

اگر روند جایگذاری تا x_0 را انجام دهیم، ضریب x_0 حاصلضرب تمام آلفاها از یک تا t خواهد بود و واریانس نویز هم متناسب با آن تغییر می‌کند.

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s \rightarrow x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\bar{\epsilon}$$

که دقیقاً معادل توزیع نرمال با میانگین و واریانس اشاره شده است بنابراین:

$$q(x_t|x_0) = N(\sqrt{\bar{\alpha}_t}x_0, (\sqrt{1 - \bar{\alpha}_t})I)$$

سؤال دوم: مزیت پیشبینی نویز

دو مزیت اصلی پیشبینی نویز ϵ بجای پیشبینی میانگین یا تصویر اصلی x_0 عبارتند از:

۱. می‌توان گفت که یکی از مهم‌ترین دلایل ثبات توزیع هدفبرای شبکه عصبی است. در حالت پیشبینی میانگین مقدار x_0 به شدت وابسته به x_t است و با جلو رفتن زمان از ۰ تا t اسکیل و دامنه مقادیر تغییر می‌کند. شبکه عصبی باید یاد بگیرد که در زمان‌های مختلف خروجی‌هایی با

مقیاس‌های متفاوت بدهد که فرآیند آموزش را دشوار می‌کند. اما در حالت پیشبینی نویز نویز هدف همیشه یک توزیع نرمال استاندارد است. یعنی صرفنظر از اینکه t چقدر است یعنی اول یک فرآیند باشیم یا آخر آن، خروجی مطلوب شبکه همیشه در یک رنج مشخص قرار دارد. این ویژگی باعث می‌شود گرادیان‌ها پایدارتر باشند و همگرایی شبکه سریع‌تر و راحت‌تر انجام شود.

۲. با استفاده از پیشبینی نویز تابع هزینه ساده‌تر و کیفیت تصویر بالاتر می‌رود. وقتی مدل را برای پیشبینی ϵ طراحی می‌کنیم، تابع هزینه که از طریق VLB بدست می‌آید ساده‌تر می‌شود. رابطه ریاضی VLB برای هر گام t شامل یک ضریب وزنی پیچیده است اما اگر ϵ را پیشبینی کنیم می‌توانیم آن ضرایب وزنی را نادیده بگیریم و از یک تابع هزینه ساده MSE استفاده کنیم. همچنین در مقاله DDPM اشاره شده که فرمول ساده شده تابع هزینه باعث می‌شود مدل تمرکز کمتری روی جزئیات بسیار ریز و نامحسوس که در t های خیلی کوچک هستند داشته باشد و تمرکز بیشتری روی ساختار کلی و محتوای تصویر بگذارد که منجر به تولید تصاویری با کیفیت بالاتر می‌شود.

سوال سوم: تحلیل تابع هزینه (VLB):

۱. الف) ما یک توزیع مرجع یا حقیقی داریم که با $q(x_{t-1}|x_t, x_0)$ نمایش داده می‌شود. این توزیع به ما می‌گوید که اگر تصویر نهایی x_t را داشته باشیم و بدانیم تصویر اولیه x_0 چه بوده است، توزیع احتمال مرحله قبل یعنی x_{t-1} چگونه باید باشد. این توزیع چون براساس توزیع‌های گوسی بنا شده فرمول بسته ریاضی دارد و قابل محاسبه است. مدل ما دسترسی به x_0 ندارد چون هدفش تولید تصویر از نویز است بنابراین سعی می‌کند توزیعی بسازد که تنها با دانستن x_t مرحله قبل را حدس بزند. ترم‌های KL در VLB معیاری برای سنجش فاصله یا اختلاف اطلاعاتی بین این دو توزیع هستند. درواقع این ترم‌ها مدل را تشویق می‌کنند تا توزیع پیشبینی شده خود را تا حد ممکن به توزیع واقعی و ایده‌آل منطبق کند. به زبان دیگر ترم‌های KL مدل را مجبور می‌کنند که در هر گام زمانی t دقیقاً همان رفتاری تقلید کند که معادلات فیزیک فرآیند انتشار در جهت معکوس بیان می‌کنند. درواقع اگر KL صفر شود یعنی مدل دقیقاً توانسته دینامیک معکوس نویز را یاد بگیرد.

۲. ب) در مقاله DDPM نشان داده شده است که استفاده از فرمول دقیق VLB منجر به نتایج بصری ضعیف‌تری نسبت به نسخه ساده شده می‌شود. در سوال قبلی گفتیم که پیشبینی نویز بجای میانگین منجر به رسیدن به تابع زیان MSE می‌شود. در واقع از آنجا که هر دو توزیع q و p_θ گوسی هستند، محاسبه D_{KL} رد آنها فرم بسته‌ای دارد:

$$D_{KL}(N(\mu_q, \Sigma) || N(\mu_\theta, \Sigma)) \propto ||\mu_q - \mu_\theta||^2$$

که در آن می‌توان میانگین را به صورت تابعی از نویز پیشبینی شده بازنویسی کرد و مینیمم کردن KL دقیقاً معادل مینیمم کردن خطای نویز $\|\epsilon - \epsilon_\theta\|^2$ می‌باشد.

همچنین در فرمول اصلی VLB هر ترم L_{t-1} دارای یک کوفیشنت است که به β_t و واریانس‌ها بستگی دارد. در فرمول اصلی هدف بهینه‌سازی $Log Likelihood$ است. از منظر ریاضی $likelihood$ اهمیت زیادی به جزئیات ریز و فرکانس بالا می‌دهد که ممکن است برای چشم انسان دارای اهمیت نباشد. اما در لاس ساده بادر نظر گرفتن ضرایب VLB و وزن یکسان در نظر گرفتن برای همه t ها یا به عبارت دیگر همان MSE ساده در عمل یک بازتعریف وزن مناسب را انجام می‌دهیم. این کار باعث می‌شود تاثیر ترم‌هایی که مربوط به یک t کوچک هستند کاهش بیاید و بجای آن تمرکز مدل روی t های بزرگ یعنی جایی که ساختار کلی تصویر شکل می‌گیرد افزایش بیابد.

همچنین باید به این نکته اشاره کرد که در تابع هزینه VLB ترم L_T مربوط به واگرایی میان $q(x_T, x_0), p(x_T)$ می‌باشد. چون x_T نویز خالص است و پارامتر یادگیرنده ندارد، این ترم ثابت است بنابراین گرادیان آن صفر می‌شود و تاثیری در آموزش شبکه ندارد. ترم L_0 نیز معمولاً به یک لاس بازسازی پیکسلی جداگانه تبدیل می‌شود که عملاً در لاس MSE با فرمول کلی یکپارچه شده و نیازی به محاسبه جداگانه ندارد.

سوال چهارم: نمونه برداری DDIM:

۱. الف) در حالت استاندارد DDPM فرآیند معکوس‌سازی یک فرآیند تصادفی است چرا که در هر گام ما یک نویز تصادفی گاوسی به تصویر اضافه می‌کنیم. در فرمول نمونه‌گیری DDIM یک ضریب σ_t وجود دارد که میزان تصادفی بودن فرآیند معکوس‌سازی را کنترل می‌کند و این ضریب مستقیماً با η ارتباط دارد:

$$\sigma_t = \eta \sqrt{\frac{1 - \alpha_{t-1}}{1 - \alpha_t}} \sqrt{1 - \frac{\alpha_t}{\alpha_{t-1}}}$$

زمانی که مقدار η برابر با یک است فرمول به فرمول DDPM استاندارد تبدیل می‌شود و زمانی که این مقدار برابر با صفر است ترم نویز تصادفی حذف می‌شود. به عبارت دیگر یعنی اگر فرآیند نمونه‌گیری با یک نویز اولیه ثابت چندین بار اجرا شود، دقیقاً یک تصویر خروجی را دریافت خواهیم کرد.

۲. ب) دلیل اصلی کندی DDPM این است که فرآیند معکوس آن یک تقریب از فرآیند انتشار واقعی است که فقط برای گام‌های بسیار کوچک معتبر است و اگر گام‌های زمانی را بزرگ بگیریم خطای تقریب گاوسی بالا می‌رود و تصویر خراب می‌شود. در DDIM از آنجا که فرآیند یک فرآیند قطعی

است، مسیر حرکت از نویز به تصویر بسیار هموارتر است و مستقیم‌تر از فرآیند *random walk* موجود در *DDPM* است. همچنین در فرمول *DDIM* نمونه‌گیری مستقیماً بر اساس تخمین x_0 انجام می‌شود که به ما اجازه می‌دهد پرش زمانی داشته باشیم یعنی مدل می‌تواند فرض کند که x_0 پیش‌بینی شده در بازه‌های زمانی طولانی‌تری معتبر است و مثلاً مستقیم از گام زمانی هزارم به ۹۰۰ بپرد، بدون اینکه خطای *Discretization* زیادی ایجاد شود. به همین دلیل *DDIM* معمولاً می‌تواند با ۵۰ تا ۱۰۰ گام تصاویری با کیفیتی تولید کند که *DDPM* برای تولید آن‌ها به ۱۰۰۰ گام نیاز دارد.

سوال پنجم: هدایت بدون دسته‌بند (CFG):

۱. الف) ایده کلیدی این روش این است که به جای استفاده از یک مدل کلسیفایر جداگانه برای هدایت فرآیند تولید تصویر، از خود مدل تولیدکننده برای هدایت استفاده کنیم. در این روش یک مدل دیفیوژن واحد آموزش داده می‌شود که همزمان قادر به تولید شرطی و غیرشرطی است. در حین آموزش این مدل، برچسب‌های شرطی مثل متن یا کلاس تصویر با یک احتمال مشخص حذف می‌شوند (Null token جایگزین می‌شود) تا مدل یاد بگیرد بدون شرط هم تصویر تولید کند. نحوه نمونه‌برداری نیز به این صورت است که در زمان تولید تصویر، خروجی نهایی ترکیبی خطی از پیش‌بینی مدل شرطی $\epsilon_\theta(z, c)$ و مدل غیر شرطی $\epsilon_\theta(z)$ است:

$$\tilde{\epsilon}_\theta(z, c) = (1 + \omega)\epsilon_\theta(z, c) - \omega\epsilon_\theta(z)$$

همچنین به آن classifier-free گفته می‌شود چرا که در روش‌های قبلی مثل Classifier Guidance نیاز بود که یک شبکه کلسیفایر جداگانه روی تصاویر نویزی آموزش داده شود تا گرادینان $\nabla_{z_t} \log p(c|z_t)$ را محاسبه کند و مدل را هدایت نماید. اما در این روش هیچ شبکه کلسیفایر جداگانه‌ای وجود ندارد و نیازی به محاسبه گرادینان کلسیفایر نیست. هدایت فرآیند تنها با استفاده از خود مدل دیفیوژن و ترکیب امتیازهای شرطی و غیرشرطی انجام می‌شود.

۲. ب) ضریب رهانمایی ω نقش کنترل‌کننده تعادل بین کیفیت و تنوع را دارد. با افزایش ضریب رهانمایی کیفیت بصری تک تک نمونه‌ها و انطباق آن‌ها با شرط (کلاس یا متن) افزایش می‌یابد. مقاله نشان می‌دهد که این کار باعث بهبود معیار Inception Score می‌شود. به طور شهودی این کار باعث می‌شود مدل روی نواحی‌ای از توزیع احتمال تمرکز کند که احتمال حضور آن کلاس خاص بسیار زیاد است و ویژگی‌های تصویر برجسته‌تر و گاهی اشباع‌شده‌تر می‌شوند. همچنین وقتی این ضریب افزایش پیدا کند، تنوع نمونه‌های تولیدی به شدت کاهش می‌یابد. با زیاد شدن ω مدل تمایل می‌کند که تنها

شبیه‌ترین و مطمئن‌ترین تصاویر مربوط به آن کلاس را تولید کند و از تولید تصاویر متنوع یا حالت‌های خاص و نادر اجتناب کند که به نوعی mode collapse محسوب می‌شود.

بخش دوم: بخش عملی

زیربخش اول: مقدمه‌ای بر مدل‌های دیفیوژن (DDIM و DDPM):

مرحله اول: پیاده‌سازی برنامه‌ریزی نویز (Variance Scheduler)

با توجه به مقادیر گفته شده Linear Scheduler را اجرا می‌کنیم که نتیجه آن به صورت زیر است:

```
Beta schedule shape: torch.Size([1000])
Beta range: 9.999999747378752e-05 to 0.019999999552965164
```

شکل ۱. نتیجه Linear Scheduler

مرحله دوم: پیاده‌سازی تابع perturb_input

با استفاده از Reparameterization Trick تابع را تکمیل کرده و x_t و نویز را محاسبه می‌کنیم که قطعه کد آن به شکل زیر است:

```
def perturb_input(x0, t):
    noise = torch.randn_like(x0).to(device)

    sqrt_ab_t_batch = sqrt_ab_t[t].to(device)
    sqrt_one_minus_ab_t_batch = sqrt_one_minus_ab_t[t].to(device)

    sqrt_ab_t_batch = sqrt_ab_t_batch.view(-1, 1, 1, 1)
    sqrt_one_minus_ab_t_batch = sqrt_one_minus_ab_t_batch.view(-1, 1, 1, 1)

    x_t = (sqrt_ab_t_batch * x0) + (sqrt_one_minus_ab_t_batch * noise)

    return x_t, noise
```

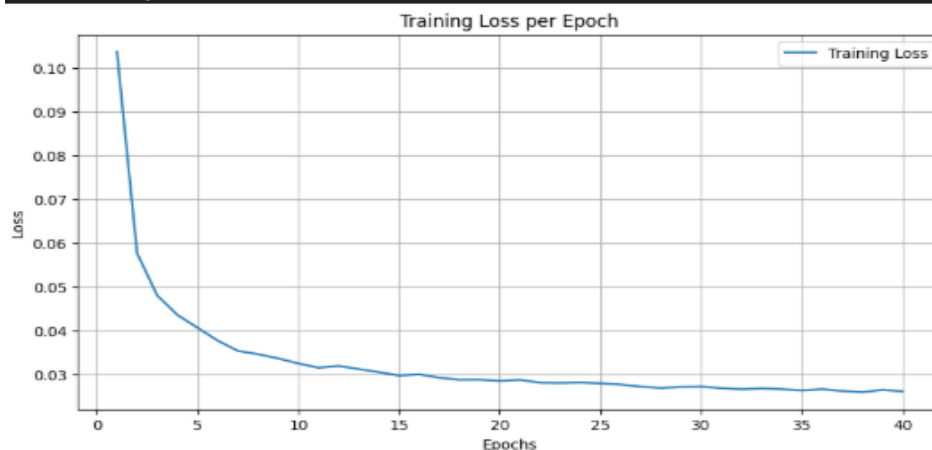
مرحله سوم: حلقه آموزش

در حلقه آموزش مراحل گفته شده در متن تمرین را پیاده‌سازی می‌کنیم و فرآیند آموزش را در epoch ۴۰ به انجام می‌رسانیم. نمودار خطای گفته‌شده بر حسب اپیاک در فرآیند آموزش به شکل زیر است:

```

Epoch 38/40
Loss: 0.0304: 100% | 600/600 [01:12<00:00, 8.231t/s]
Average Loss for Epoch 38: 0.02600
Epoch 39/40
Loss: 0.0166: 100% | 600/600 [01:12<00:00, 8.231t/s]
Average Loss for Epoch 39: 0.02656
Epoch 40/40
Loss: 0.0208: 100% | 600/600 [01:12<00:00, 8.231t/s]
Average Loss for Epoch 40: 0.02618
Model saved at epoch 39

```



شکل ۲. نمودار خطا در طول آموزش به همراه وضعیت ۳ ایپاک آخر

مرحله چهارم: پیاده‌سازی نمونه‌برداری (Sampling)

الگوریتم‌های نمونه‌برداری معکوس برای DDIM و DDIM را پیاده می‌کنیم و در نظر داریم که در DDIM در این فرآیند طول گام ۵۰ است که ۵۰ برابر گام‌های واحد در نمونه‌برداری DDPM است. نتایج حاصله از پیاده‌سازی این نمونه‌برداری و نتایج FID در ادامه آورده شده‌است:



شکل ۳. نمونه‌برداری DDPM



شکل ۴. نمونه برداری DDIM

418 ms $\pm$ 0 ns per loop (mean $\pm$ std. dev. of 1 run, 1 loop each)
 22.4 s $\pm$ 0 ns per loop (mean $\pm$ std. dev. of 1 run, 1 loop each)

شکل ۵. مقایسه سرعت DDPM (پایین) و DDIM (بالا)

```
--- Calculating FID for DDPM (Standard) ---
Processing real images for DDPM (Standard)...
Generating 1000 fake images using DDPM (Standard)...
Computing score...000
>> FID Score for DDPM (Standard): 102.1169

--- Calculating FID for DDIM (Accelerated) ---
Processing real images for DDIM (Accelerated)...
Generating 1000 fake images using DDIM (Accelerated)...
Computing score...
>> FID Score for DDIM (Accelerated): 242.4402

===== FINAL RESULTS =====
DDPM FID: 102.1169
DDIM FID: 242.4402
```

شکل ۶. نتایج FID Score برای هر دو حالت

همانطور که مشاهده می‌شود روش DDIM حدوداً ۵۰ برابر سریعتر از DDPM در نمونه برداری عمل کرده است اما کیفیت تصاویر نمونه برداری شده به وسیله این روش پایین تر از روش دیگر است که نتیجه FID Score نیز گواه این نکته است. البته باید این نکته را در نظر گرفت که در این پیاده سازی به دو دلیل FID score برای

هر دو روش خیلی جالب نیست و فاصله دو روش نیز بیشتر از انتظار است. اولین نکته‌ای که باید به آن توجه کرد این است که epoch ۴۰ زمان بسیار کمی برای آموزش یک مدل diffusion است و برای رسیدن به نتایج بهتر باید طول دوره آموزش را چند برابر کنیم. نکته بعدی این است که معمولا FID Score با تولید بین ۱۰۰۰۰ تا ۵۰۰۰۰ تصویر بررسی می‌شود که به دلیل بیش از حد طولانی شدن فرآیند از ۱۰۰۰ تصویر استفاده کرده‌ایم. با افزایش تعداد تصاویر نیز اختلاف بین دو امتیاز با فرض آموزش کافی کاهش خواهد یافت.

زیربخش دوم: Stable Diffusion و تکنیک DreamBooth:

مرحله اول: آماده‌سازی داده‌ها

برای این بخش سوژه انتخابی سگ‌های هاسکی هستند که در کلاس درس نیز جزو سوژه‌های انتخابی به منظور یادگیری شرطی بودند. به این منظور ۵ تصویر از این سگ‌ها آپلود می‌کنیم که در آن‌ها بدن سگ کاملا مشخص است اما سعی شده پس‌زمینه متفاوت باشد تا مدل به خود سگ بیشتر از محیط اطراف آن توجه کند. همچنین کد موجود پوشه instance_data را در خروجی محیط Kaggle قرار می‌داد که در آن‌جا نمی‌شود بصورت دستی چیزی آپلود کرد بنابراین ۵ تصویر بصورت یک دیتاست در کگل فراخوانی شدند. همچنین چون سوژه انتخابی سگ بوده نیازی به تغییر متغیرهای prompt نبود.



شکل ۷. هاسکی شماره ۱



شکل ۸. هاسکی شماره ۲



شکل ۹. هاسکی شماره ۳



شکل ۱۰. هاسکی شماره ۴



شکل ۱۱. هاسکی شماره ۵

مرحله دوم: تکمیل کلاس دیتاست:

در این مرحله تصاویر باید به اندازه 512×512 تبدیل شوند و بین ۱ و ۱- نرمالایز شوند. همچنین متن‌ها باید توکنایز شوند تا پرامپت دریافت شود. با درست کار کردن مراحل بعدی یعنی این مرحله نیز به خوبی کار می‌کند.

مرحله سوم: تولید تصاویر کلاس

با استفاده از مدل اولیه تعداد ۱۰۰ تصویر از سگ‌های معمولی را تولید کرده و در پوشه `class_data` ذخیره می‌کنیم. نمونه‌ای از تصاویر تولیدی به صورت زیر است:



شکل ۱۲. نمونه‌ای از تصاویر تولید شده `class_data`

مرحله چهارم: حلقه آموزش

الگوریتم را مطابق با مراحل نوشته شده در توضیحات قطعه کد پیاده‌سازی می‌کنیم و وزن‌های LoRA را آپدیت می‌کنیم تا برای مرحله بعدی آماده شود.

Algorithm Detail: Standard Diffusion Training:

1. **Latents:** Encode image x to latent $z_0 = \mathcal{E}(x)$.
2. **Noise:** Sample gaussian noise $\epsilon \sim \mathcal{N}(0, I)$ and timestep t .
3. **Forward Diffusion:** Add noise to get $z_t = \sqrt{\alpha_t}z_0 + \sqrt{1 - \alpha_t}\epsilon$.
4. **Prediction:** The model predicts the noise: $\epsilon_\theta(z_t, t, c)$, where c is the text embedding.
5. **Loss:** $L = \|\epsilon - \epsilon_\theta(z_t, t, c)\|^2$.

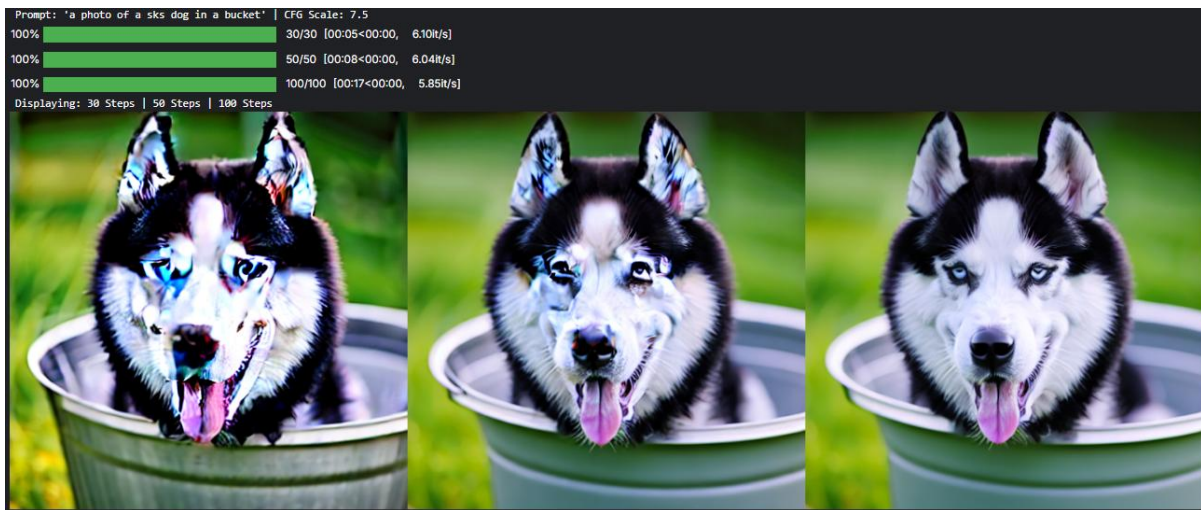
شکل ۱۳. الگوریتم آموزش مدل

```
Starting training...
Steps: 100% ██████████ 1000/1000 [27:50<00:00, 1.10s/it]
Training finished!
Model and Text Encoder safely saved to ./lora-dreambooth-model
```

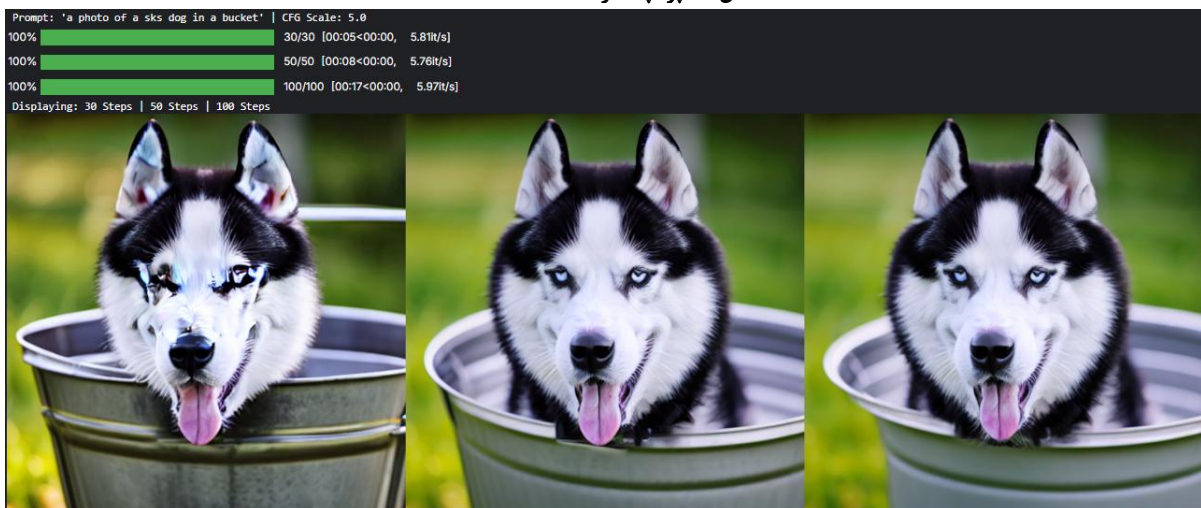
شکل ۱۴. فرآیند آموزش

مرحله پنجم: تست و استنتاج

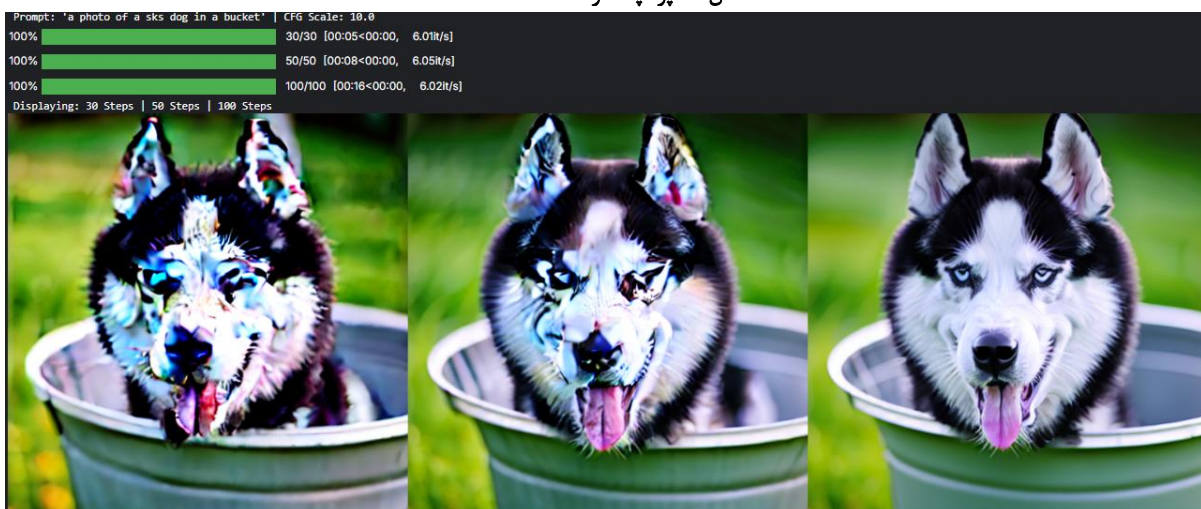
با پرامپت‌های مختلف نتیجه حاصله را ارزیابی میکنیم:



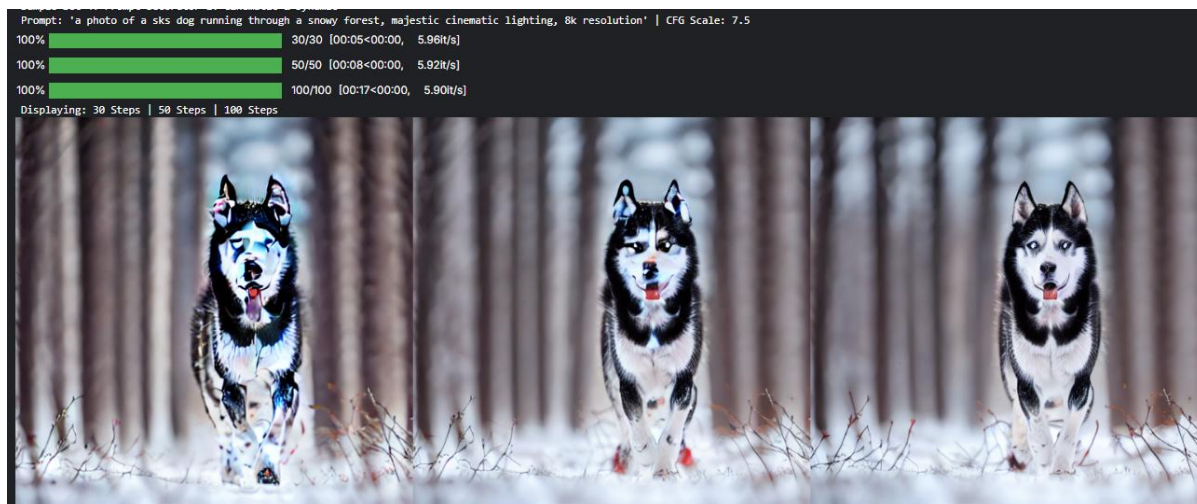
شکل ۱۵. پرامپت در CFG=7.5



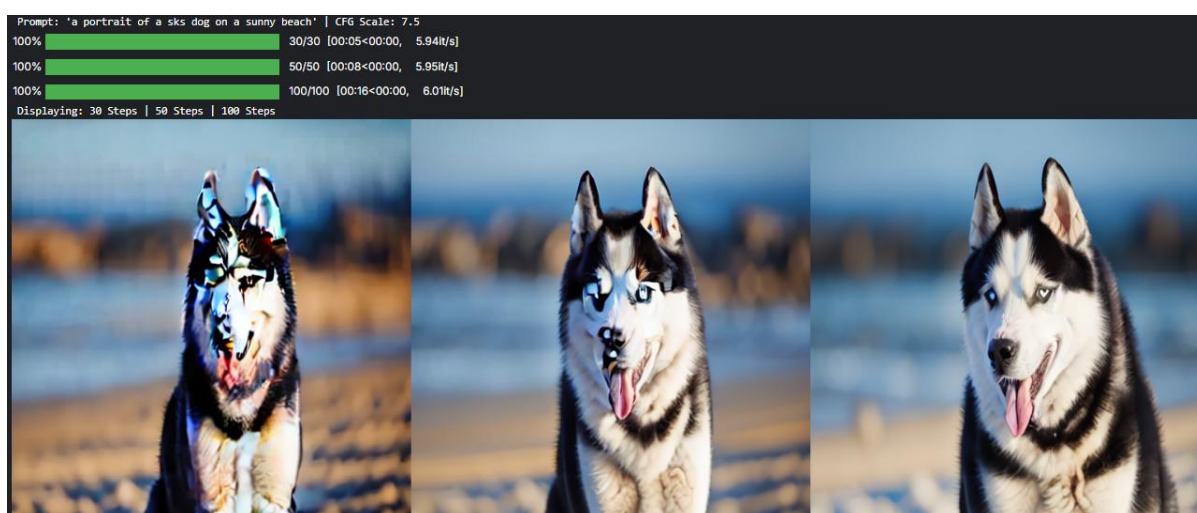
شکل ۱۶. پرامپت در CFG=5



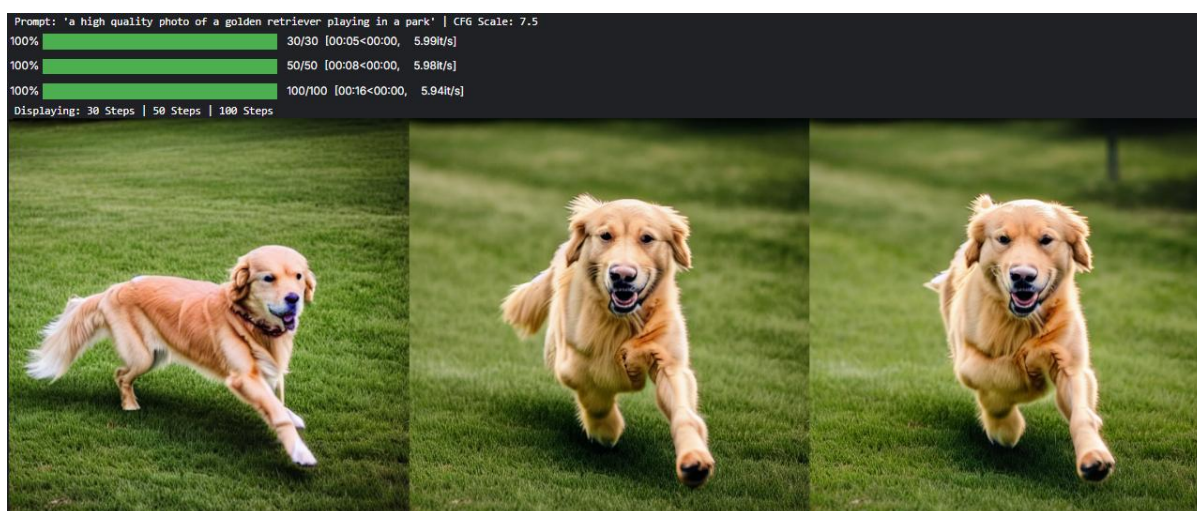
شکل ۱۷. پرامپت در CFG=10



شکل ۱۸. پرامپتی دیگر در $CFG=7.5$



شکل ۱۹. پرامپت سوم در $CFG=7.5$



شکل ۲۰. تولید سگ غیر هاسکی در $CFG=7.5$

با توجه به این تصاویر به طو کلی می توان گفت و مورد انتظار نیز می باشد که با افزایش تعداد step ها تمرکز مدل بر روی prompt افزایش پیدا کرده و کیفیت عکس نیز بهتر شده است. درواقع تصاویر از حالت

پیکسلی خود درآمده و شکل واقعی تری به خود می گیرند. با افزایش میزان CFG نیز کنتراست تصاویر بالاتر می رود و sharp تر می شوند که گاهی بیش از حد است و باعث غیر واقعی جلوه دادن تصویر می شود. همچنین در CFG بالاتر تصاویر در تعداد گام پایین تر کیفیت بدتری داشتند. در کل می توان گفت که شناسایی توکن خاص توسط این مدل تا حد بسیار خوبی صورت پذیرفته و همچنین مانع فراموشی دانسته های قبلی نیز نشده چون تصویر سگ دیگر به خوبی تولید شده است. همچنین حالت های متخلف گفته شده به مدل برای تولید تا حد خوبی رعایت شده اند و انعطاف مدل نیز بد نبوده است.

سوال دوم: فلو مچینگ و سری های زمانی

بخش اول: سوالات تئوری

سوال اول: میدان برداری:

۱. الف) در مدل های Flow Matching هدف یادگیری یک مسیر احتمالاتی است که توزیع نویز ساده مثلاً نویز گاوسی را به توزیع داده های پیچیده متصل می کند. میدان برداری $v_\theta(x, t)$ می تواند نقش های متفاوتی داشته باشد. میدان برداری در واقع سرعت یا جهت لحظه ای حرکت نمونه ها را در هر نقطه از زمان t و فضا x مشخص می کند. این میدان برداری یک ODE به فرم زیر را تعریف می کند:

$$\frac{dx}{dt} = v_\theta(x, t)$$

برای تولید داده از یک نمونه نویز $x_0 \sim p_0$ شروع می کنیم و با حل این ODE از زمان $t=0$ تا $t=1$ ، نمونه را در امتداد خطوط جریان میدان برداری حرکت می دهیم تا به فضای داده p_1 برسیم. برخلاف دیفیوژن که مسیرها تصادفی هستند، FM سعی دارد میدان برداری را طوری یاد بگیرد که نمونه ها را در مسیرهای مستقیم تر و بهینه تر جابه جا کند.

۲. ب) مدل های دیفیوژن بر پایه SDE یا معادلات دیفرانسیل تصادفی بنا شده اند و با اضافه کردن تدریجی نویز و یادگیری حذف آن کار می کنند. اما FM بر پایه ODE می باشد و مستقیماً فیلد سرعت انتقال جرم را مدل سازی می کند. در دیفیوژن مدل معمولاً تلاش می کند Score Function یا مقدار نویز اضافه شده را تخمین بزند. در FM هدف اصلی یادگیری مستقیم Vector Field است که بین دو توزیع درونیابی می کند.

مهم ترین تفاوت در کارایی و سرعت نمونه برداری است. مدل های دیفیوژن به دلیل ماهیت تصادفی و خمیدگی مسیرها معمولاً به گام های زیادی برای حل معادله نیاز دارند تا کیفیت حفظ شود. در مقابل FM مسیرهایی بسیار صاف و مستقیم ایجاد می کند که اجازه می دهد با تعداد گام های بسیار کم به خروجی با کیفیت رسید.

این دو مدل زمانی به هم شبیه تر می شوند که دیفیوژن مدل را از طریق Probability Flow ODE بررسی کنیم. هر مدل دیفیوژن یک ODE معادل دارد که تکامل توزیع احتمالاتی را توصیف می کند. اگر میدان برداری در FM را دقیقاً برابر با فیلد استخراج شده از Score Function در دیفیوژن قرار دهیم با در نظر گرفتن ضرایب Noise Schedule، این دو مدل از نظر ریاضی معادل

یکدیگر خواهند بود. به عبارت دیگر Diffusion را می‌توان حالتی خاص از Flow Matching دانست که در آن مسیرهای انتقال لزوماً مستقیم نیستند و توسط Schedule نويز دیکته می‌شوند.

سوال دوم: مسیرهای خطی:

۱. الف) در مدل‌های FM ساخت داده‌های میانی x_t از طریق یک درونیابی خطی بین داده واقعی و نويز انجام می‌شود:

$$x_t = tx_1 + (1 - t)x_0$$

این انتخاب سادگی و صراحت ریاضی دارد و به همین دلیل است که انتخاب جذابی است. میدان برداری که نشان‌دهنده سرعت تغییرات در طول این مسیر است با یک مشتق‌گیر ساده نسبت به زمان به دست می‌آید:

$$\frac{dx_t}{dt} = x_1 - x_0$$

این یعنی در فرمولاسیون شرطی سرعت حرکت در طول مسیر ثابت است. مدل به جای یادگیری مسیرهای پیچ‌دار و تصادفی مثل SDE در مدل‌های دیفیوژن، تنها کافی است یک جهت ثابت یعنی $x_1 - x_0$ را پیش‌بینی کند. اما مسیر خطی همواره بهینه نیست. داده‌های پیچیده در دنیای واقعی به صورت یکنواخت در فضای اقلیدسی پخش نشده‌اند بلکه روی منی‌فولدهای پیچیده و غیر خطی قرار دارند. یک خط راست اقلیدسی بین نويز و داده ممکن است مدل را از فضایی کاملاً بی‌معنی و خارج از منی‌فولد عبور دهد. در این موارد مسیرهای Geodesic روی خود منی‌فولد بسیار بهینه‌تر از یک خط راست ساده در فضای مبدا می‌باشند. جدای از آن حتی اگر مسیر حرکت یک نمونه خاص خطی باشد، اگر توزیع نويز و توزیع داده‌ها به درستی باهم جفت نشده باشند، خطوط حرکت نمونه‌های مختلف با یکدیگر تقاطع پیدا می‌کنند. وقتی مسیرها متقاطع شوند، میدان برداری نهایی که شبکه باید یاد بگیرد، دیگر خطی و ساده نیست بلکه تبدیل به میانگینی پیچیده و غیرخطی از جهت‌های مختلف در نقطه تقاطع می‌شود. این مشکل در برخی رویکردهای FM، با بهینه‌سازی نحوه جفت کردن x_1 و x_0 تا حدودی کاهش می‌یابد اما فرض خطی بودن مطلق فضا همچنان یک ساده‌سازی محسوب می‌شود.

۲. ب) به لطف مسیر خطی، تابع هزینه در Conditional Flow Matching از درگیری با شبیه‌سازی‌های پیچیده و انتگرال‌گیری‌های زمانبر رها می‌شود. از آنجا که میدان برداری هدف برای یک جفت مشخص (x_0, x_1) برابر با مقدار ثابت $x_1 - x_0$ می‌باشد، تابع هزینه با یک خطای MSE ساده تبدیل می‌شود:

$$L(\theta) = E_{t, x_0, x_1} \left[\|v_\theta(x_t, t) - (x_1 - x_0)\|^2 \right]$$

در مدل‌های دیفیوژن استاندارد در زمان‌هایی که نویز بسیار زیاد است یعنی t های نزدیک به پایان فرآیند فوروارد، پیش‌بینی دقیق سیگنال نویز یا Score Function می‌تواند دچار عدم ثبات و واریانس بالا شود. اما در FM با مسیر خطی، شبکه همواره در حال پیش‌بینی یک سرعت ثابت و کراندار است. این مسئله از انفجار یا محو شدن گرادیان‌ها جلوگیری کرده و واریانس آموزش را به شدت کاهش می‌دهد. این پایداری تنها مختص به زمان آموزش نیست. زمانی که مدل یاد می‌گیرد میدان‌های برداری هموارتر و مستقیم‌تری تولید کند، در زمان استنتاج یا تولید داده جدید، حل‌کننده‌های معادلات دیفرانسیل می‌توانند با گام‌های بسیار بزرگ‌تر و با خطای برشی (Truncation Error) بسیار کم‌تری حرکت خود را انجام دهند.

سوال سوم: تابع زیان

$$q(x_t|x_0) \sim N(x_0, \sigma_t^2 I)$$

$$L = E_{t, x_0, x_t} \left[\left\| s_\theta(x_t, t) - \nabla_{x_t} \log q(x_t|x_0) \right\|_2^2 \right], q(x_t|x_0) = \frac{1}{(2\pi\sigma_t^2)^{\frac{d}{2}}} \exp\left(-\frac{\|x_t - x_0\|^2}{2\sigma_t^2}\right)$$

$$\rightarrow \log q(x_t|x_0) = -\frac{d}{2} \log(2\pi\sigma_t^2) - \frac{\|x_t - x_0\|^2}{2\sigma_t^2}$$

$$\rightarrow \nabla_{x_t} \log q(x_t|x_0) = \nabla_{x_t} \left(-\frac{1}{2\sigma_t^2} (x_t - x_0)^T (x_t - x_0) \right)$$

$$\rightarrow \nabla_{x_t} \log q(x_t|x_0) = -\frac{1}{2\sigma_t^2} \times 2(x_t - x_0)$$

$$\rightarrow \nabla_{x_t} \log q(x_t|x_0) = -\frac{x_t - x_0}{\sigma_t^2} = \frac{x_0 - x_t}{\sigma_t^2}$$

$$\rightarrow L = E_{t, x_0 \sim p_{data}, \epsilon \sim N(0, I)} \left[\left\| s_\theta(x_t, t) - \frac{x_0 - x_t}{\sigma_t^2} \right\|_2^2 \right]$$

بخش دوم: سوالات عملی

ابتدا باید دادگان را طبق توضیحات داده‌شده برای مراحل بعد آماده کنیم. البته باید به این نکته توجه داشت که ابتدا باید دادگان ترین و تست را از هم جدا کرد و سپس نرمال‌سازی را بر اساس دادگان آموزش انجام داد و در غیر این صورت data leakage رخ داده است چرا که قرار است آینده را بررسی کنیم و در صورت نرمال‌سازی روی کل دادگان تاثیر دادگان آینده در میانگین و واریانس فعلی لحاظ شده و پیش‌بینی را به سمت داده واقعی سوق می‌دهد که با وجود اینکه نتیجه را بهتر می‌کند اما این یک خطاست و در

پیش‌بینی واقعی ضعف مدل نمایان خواهد شد. همچنین یک مدل Flow Matching نسبتاً معمولی برای اجرای این بخش در نظر گرفته شده که خلاصه معماری پیاده‌شده در زیر آمده‌است:

```

Downloading data...
Total log-returns calculated: 3521
Total overlapping sequences created: 3458
Train set shape: (3112, 64)
Test set shape: (346, 64)
Global Train Mean: 0.000480, Global Train Std: 0.010951
Data preparation completed successfully.

```

شکل ۲۱. لود و تقسیم دیتاست

```

--- Model Summary ---
=====
Layer (type:depth-idx)      Input Shape      Output Shape     Param #
=====
FlowMatchingModel          [512, 64]        [512, 64]        --
├─SinusoidalTimeEmbedding: 1-1 [512, 1]         [512, 128]       --
├─Sequential: 1-2           [512, 128]       [512, 256]       --
│   └─Linear: 2-1            [512, 128]       [512, 256]       33,024
│   └─SiLU: 2-2              [512, 256]       [512, 256]       --
│   └─Linear: 2-3            [512, 256]       [512, 256]       65,792
├─Linear: 1-3               [512, 64]        [512, 256]       16,640
├─SiLU: 1-4                 [512, 256]       [512, 256]       --
├─ModuleList: 1-9           --                --                (recursive)
│   └─ResBlock: 2-4          [512, 256]       [512, 256]       --
│       └─Linear: 3-1        [512, 256]       [512, 256]       65,792
│       └─SiLU: 3-2         [512, 256]       [512, 256]       --
│       └─Dropout: 3-3      [512, 256]       [512, 256]       --
│       └─Linear: 3-4       [512, 256]       [512, 256]       65,792
├─SiLU: 1-6                 [512, 256]       [512, 256]       --
├─ModuleList: 1-9           --                --                (recursive)
│   └─ResBlock: 2-5          [512, 256]       [512, 256]       --
│       └─Linear: 3-5        [512, 256]       [512, 256]       65,792
│       └─SiLU: 3-6         [512, 256]       [512, 256]       --
│       └─Dropout: 3-7      [512, 256]       [512, 256]       --
│       └─Linear: 3-8       [512, 256]       [512, 256]       65,792
├─SiLU: 1-8                 [512, 256]       [512, 256]       --
├─ModuleList: 1-9           --                --                (recursive)
│   └─ResBlock: 2-6          [512, 256]       [512, 256]       --
│       └─Linear: 3-9        [512, 256]       [512, 256]       65,792
│       └─SiLU: 3-10        [512, 256]       [512, 256]       --
│       └─Dropout: 3-11     [512, 256]       [512, 256]       --
│       └─Linear: 3-12      [512, 256]       [512, 256]       65,792
├─SiLU: 1-10                [512, 256]       [512, 256]       --
└─Linear: 1-11              [512, 256]       [512, 64]        16,448
=====
Total params: 526,656
Trainable params: 526,656
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 269.65
=====
Input size (MB): 0.13
Forward/backward pass size (MB): 9.70
Params size (MB): 2.11
Estimated Total Size (MB): 11.94
=====

```

شکل ۲۲. خلاصه‌ای از مدل

پس از آن به آموزش مدل می‌پردازیم و بهترین مدل را ذخیره می‌کنیم.

مرحله اول: ثبت و بررسی تابع هزینه در طول آموزش

```
Starting Training on cuda for 100 epochs...  
Epoch 001/100 | Train Loss: 2.00362 | Test Loss: 2.06029 | LR: 0.000800  
Epoch 002/100 | Train Loss: 1.98278 | Test Loss: 2.06436 | LR: 0.000800  
-> EarlyStopping counter: 1 out of 20  
Epoch 003/100 | Train Loss: 1.96550 | Test Loss: 2.05544 | LR: 0.000800  
Epoch 004/100 | Train Loss: 1.95499 | Test Loss: 2.04824 | LR: 0.000800  
Epoch 005/100 | Train Loss: 1.93666 | Test Loss: 2.03818 | LR: 0.000800  
Epoch 006/100 | Train Loss: 1.91262 | Test Loss: 2.03425 | LR: 0.000800  
Epoch 007/100 | Train Loss: 1.92016 | Test Loss: 2.03489 | LR: 0.000800  
-> EarlyStopping counter: 1 out of 20  
Epoch 008/100 | Train Loss: 1.87007 | Test Loss: 2.03236 | LR: 0.000800  
Epoch 009/100 | Train Loss: 1.88008 | Test Loss: 2.00595 | LR: 0.000800  
Epoch 010/100 | Train Loss: 1.83345 | Test Loss: 2.00337 | LR: 0.000800
```

شکل ۲۳. وضعیت خطای آموزش و ارزیابی در ده ایپاک اول

```
Epoch 086/100 | Train Loss: 1.52244 | Test Loss: 1.71905 | LR: 0.000050  
-> EarlyStopping counter: 10 out of 20  
Epoch 087/100 | Train Loss: 1.53636 | Test Loss: 1.74657 | LR: 0.000050  
-> EarlyStopping counter: 11 out of 20  
Epoch 088/100 | Train Loss: 1.53393 | Test Loss: 1.75452 | LR: 0.000050  
-> EarlyStopping counter: 12 out of 20  
Epoch 089/100 | Train Loss: 1.51731 | Test Loss: 1.71241 | LR: 0.000050  
-> EarlyStopping counter: 13 out of 20  
Epoch 090/100 | Train Loss: 1.52369 | Test Loss: 1.74999 | LR: 0.000050  
-> EarlyStopping counter: 14 out of 20  
Epoch 091/100 | Train Loss: 1.53909 | Test Loss: 1.77513 | LR: 0.000050  
-> EarlyStopping counter: 15 out of 20  
Epoch 092/100 | Train Loss: 1.54624 | Test Loss: 1.71731 | LR: 0.000050  
-> EarlyStopping counter: 16 out of 20  
Epoch 093/100 | Train Loss: 1.52583 | Test Loss: 1.71632 | LR: 0.000050  
-> EarlyStopping counter: 17 out of 20  
Epoch 094/100 | Train Loss: 1.52518 | Test Loss: 1.72668 | LR: 0.000025  
-> EarlyStopping counter: 18 out of 20  
Epoch 095/100 | Train Loss: 1.52245 | Test Loss: 1.77601 | LR: 0.000025  
-> EarlyStopping counter: 19 out of 20  
Epoch 096/100 | Train Loss: 1.54526 | Test Loss: 1.74402 | LR: 0.000025  
-> EarlyStopping counter: 20 out of 20  
-> Early Stopping Triggered! Halting training.  
Training Session Ended.  
Loaded the best model weights from the checkpoint.
```

شکل ۲۴. وضعیت خطای آموزش و ارزیابی در ده ایپاک آخر

مرحله دوم: رسم نمودار Loss آموزش و آزمون

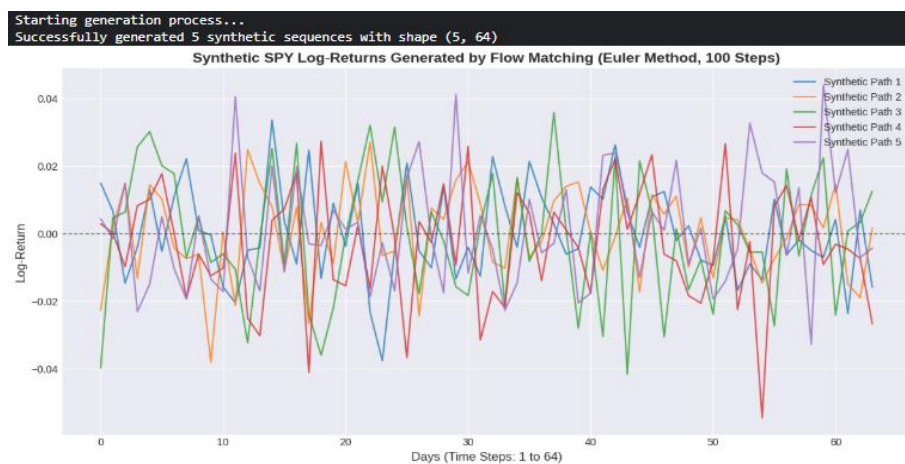


شکل ۲۵. نمودار خطای آموزش و ارزیابی در طول آموزش

با مشاهده این نمودار می‌توان گفت که مدل دچار اندکی overfitting شده است چرا که از محدوده ایپاک چهارم خطای آموزش در حال کاهش بوده اما خطای ارزیابی کاهش پیدا نکرده است. در واقع داده‌های بورس به شدت دارای نویز هستند و سیگنال خالص در آن‌ها کم است و مدل در ۴۰ تا ۵۰ ایپاک اول الگوهای نسبتاً واقعی را یاد گرفته و پس از آن در حال استفاده از ظرفیت خود برای یادگیری نویز و پرش‌های اتفاقی بوده‌است. اگر میزان دیتا را با داده‌افزایی افزایش می‌دادیم یا سال‌های بیشتری از داده را در اختیار داشتیم احتمالاً بهتر می‌توانستیم از ظرفیت مدل استفاده کنیم و احتمال overfitting نیز کاهش پیدا می‌کرد.

مرحله سوم: تولید داده مصنوعی (SAMPLING)

با استفاده از میدان برداری آموزش دیده تولید داده را انجام می‌دهیم.



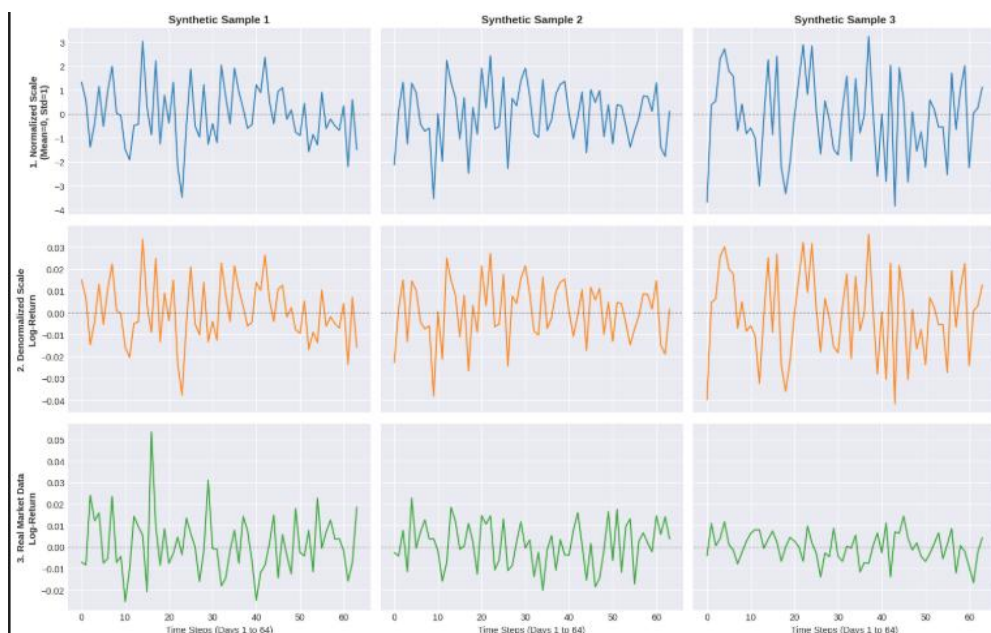
شکل ۲۶. نمونه‌ای از داده‌های تولید شده

در اینجا ما از سولور اوپلر استفاده کردیم که پایه‌ای ترین و ساده‌ترین روش عددی برای حل معادلات دیفرانسیل معمولی است. در واقع این روش در یک حلقه for یک تقریب زنی خطی گام به گام را انجام می‌دهد. مدل ما در هر گام بردار سرعت یا جهت حرکت را در هر لحظه به ما می‌دهد سپس سولور با فرمول $x = x + v * dt$ نمونه را به اندازه یک گام زمانی در آن جهت حل می‌دهد تا موقعیت جدید بدست بیاید. همچنین با دقت خوبی می‌توان گفت که افزایش گام‌های محاسبه دقت را افزایش خواهد داد و کیفیت خروجی نیز بهتر می‌شود اما هزینه محاسباتی نیز با افزایش تعداد گام‌ها افزایش پیدا خواهد کرد.

مرحله چهارم: نمایش نمونه‌های سری زمانی تولیدشده

مرحله پنجم: مقایسه کیفی سری‌های زمانی واقعی و تولیدشده

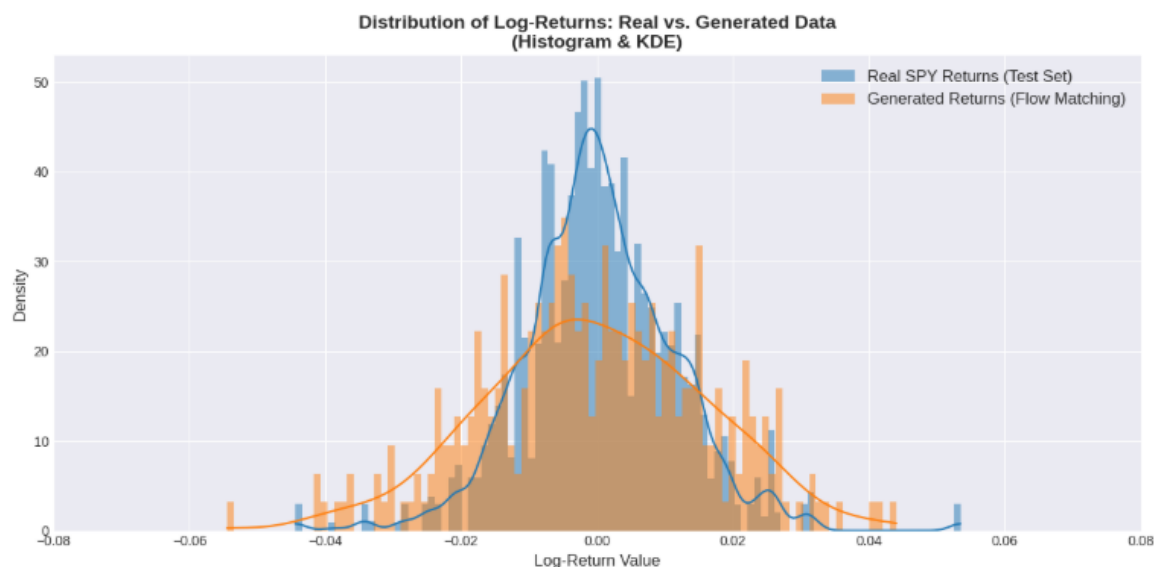
مراحل ۴ و ۵ را باهم بررسی می‌کنیم:



شکل ۲۲. نمونه‌های واقعی و تولیدشده

مشاهده می‌شود که مدل در حفظ دامنه و شکل کلی بسیار موفق عمل کرده است، اما در تولید رفتارهای افراطی بازار ضعف‌های کلاسیکی دارد. با مشاهده داده‌ها کاملاً نمایان است که مدل دامنه نوسانات و شکل کلی و ایستایی را به خوبی پیاده کرده است. همچنین در یادگیری نوسان متوسط خوب عمل کرده اما در جهش‌ها و تغییرات شدید ضعف دارد چرا که به طور کلی مدل ما و کلا مدل‌های مصنوعی رفتار بسیار محتاطانه‌تری دارند و گاهی نمی‌توانند واقعیت بازار را دنبال کنند.

مرحله ششم: مقایسه توزیع داده‌های واقعی و تولیدشده



شکل ۲۸. هیستوگرام و KDE توزیع مقادیر بازدهی لگاریتمی داده‌های واقعی و تولیدشده

مشاهده می‌شود که داده‌های واقعی دارای یک قله تیز و بلند در حوالی خط صفر هستند. این ویژگی نشان می‌دهد که در دنیای واقعی در اکثر روزهای معاملاتی نوسان بسیار کمی رخ می‌دهد و بازار آرام است. اما منحنی داده‌های تولیدشده قله‌ای بسیار کوتاه‌تر و پهن‌تر دارد یعنی اینکه مدل نتوانسته تمرکز شدید بازار حول عدد صفر را یاد بگیرد و به اشتباه تمایل دارد روزهای آرام را پرنوسان‌تر از چیزی که واقعا هست نشان دهد. همچنین داده‌های تولید توزیع تقریباً نرمالی دارند اما توزیع داده‌های واقعی اندکی با توزیع نرمال فاصله دارد. دلیل آن نیز است که ما نویز اولیه خود را از یک توزیع کاملاً نرمال استخراج می‌کنیم و شبکه عصبی نیز تابع هزینه MSE دارد بنابراین تمایل آن به سمت خروجی دادن توزیع نرمال می‌رود. بنابراین در حالت کلی می‌توان نتیجه گرفت که مدل در یادگیری مرزهای کلی و میانگین نوسانات بازار موفق بوده زیرا هر دو توزیع دقیقاً حول صفر متمرکز شده‌اند و مرزهای چپ و راست آن‌ها باهم تطابق دارد. اما در بازتولید حقایق ساختاری بازار یعنی قله بسیار تیز و دم سنگین دچار ساده سازی شده است.

مرحله هفتم: ارزیابی آماری پایه و نوسانات

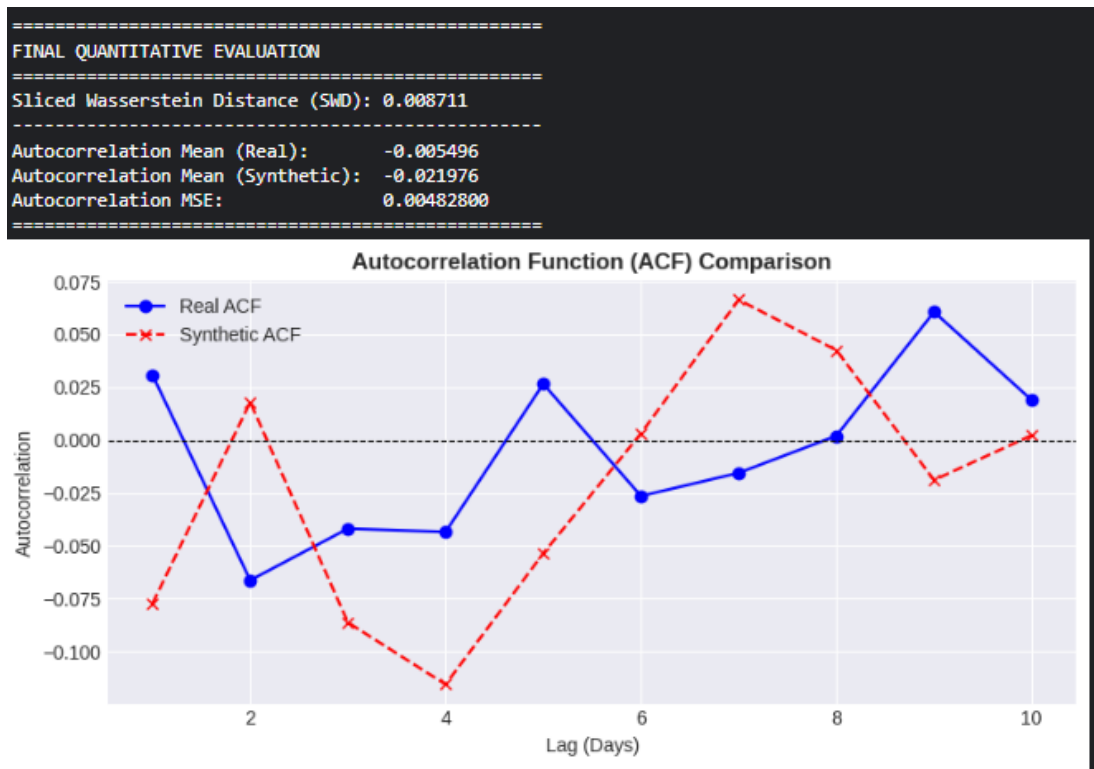
Metric	Real Data (SPY)	Generated Data
Mean	0.000400	-0.000419
Variance	0.000128	0.000268
Volatility	0.011293	0.016378

Annualized Volatility (Real): 0.1793		
Annualized Volatility (Synthetic): 0.2600		

شکل ۲۹. شاخص‌های خواسته شده

با اعداد به دست آمده مشاهده می‌شود که مدل در بازتولید دقیق شدت نوسانات موفق نبوده و شدت نوسانات را به طور قابل توجهی بیشتر از واقعیت تولید کرده است. در واقع مطابق تحلیل‌های قبل میانگین در هر دو حالت به هم نزدیک است اما میزان واریانس و نوسان به دلیل عدم در نظر گرفتن روزهای آرام و بدون نوسانات خیلی خاصی توسط مدل اندکی از واقعیت فاصله گرفته‌اند.

مرحله هشتم: ارزیابی پیشرفته ساختار توزیعی و زمانی



شکل ۳۰. ارزیابی پیشرفته

معیار SWD به زبان ساده محاسبه می‌کند که توزیع داده‌های تولیدشده چقدر باید تغییر شکل دهد تا دقیقاً روی توزیع داده‌های واقعی در فضای پیچیده ۶۴ بعدی منطبق شود. هر چه این عدد به صفر نزدیک‌تر باشد یعنی مدل توانسته است شکل کلی و ساختار هندسی داده‌ها را بهتر یاد بگیرد.

خودهمبستگی و میانگین و خطای آن نیز وابستگی زمانی یا حافظه خطی را می‌سنجند. این معیار بررسی می‌کند که آیا بازدهی یک روز ارتباطی با بازدهی روزهای گذشته دارد یا خیر. در بازارهای مالی بازدهی روزانه از قانون گام تصادفی پیروی می‌کند. یعنی فردا از امروز مستقل است و نمودار ACF باید اعدادی نزدیک به صفر را نشان دهد.

با توجه به موارد گفته شده و اعداد بدست آمده می‌توان گفت که مدل در معیار SWD بسیار خوب عمل کرده است. همچنین نمودار ACF به ما نشان می‌دهد که مدل متوجه خاصیت رندوم واک شده است. اگرچه

مدل در بازتولید شدت دقیق نوسانات و دم های پهن تا حدودی دچار ساده سازی شده اما دینامیک زمانی صحیح و ساختار مشترک چندبعدی بازار را تا حد خوبی یاد گرفته است.

مراجع

- 1: J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020
- 2: J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *International Conference on Learning Representations (ICLR)*, 2021.
- 3: J. Ho and T. Salimans, "Classifier-free diffusion guidance," arXiv preprint arXiv:2207.12598, 2022.
- 4: R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- 5: N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman, "Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- 6: E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2021.
- 7: Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *International Conference on Learning Representations (ICLR)*, 2023.